

Listado completo de los comandos de git

Contenido

1.	git init	3
2.	git clone	3
3.	git add	3
4.	git commit	3
5.	git status	3
6.	git push	4
7.	git pull	4
8.	git branch	4
9.	git checkout	4
10.	git merge	5
11.	git log	5
12.	git fetch	5
13.	git reset	5
14.	git stash	6
15.	git remote	6
16.	git diff	6
17.	git tag	6
18.	git rebase	7
19.	git cherry-pick	7
20.	git rm	7
21.	git config	7
22.	git show	7
23.	git revert	8
24.	git clean	8
25.	git blame	8
26.	git bisect	8
27.	git reflog	8
28.	git submodule	9
29.	git worktree	9
30.	git archive	9
31.	git grep	9

32.	git fsck	9
33.	git gc	10
34.	git shortlog	10
35.	git describe	10
36.	git apply	10
37.	git patch-id	10
38.	git request-pull	10
39.	git instaweb	11
40.	git help.....	11
41.	git whatchanged.....	11
42.	git count-objects	11
43.	git verify-commit.....	11
44.	git verify-tag	11
45.	git merge-base.....	12
46.	git notes.....	12
47.	git replace.....	12
48.	git rev-parse	12
49.	git symbolic-ref.....	12
50.	git update-ref	13
51.	git filter-branch	13
52.	git fast-export.....	13
53.	git fast-import	13
54.	git bundle	13
55.	git daemon	13
56.	git shell	14
57.	git svn	14
58.	git annex.....	14
59.	git lfs	14
60.	git flow.....	14
61.	git subrepo	15
62.	git absorb.....	15
63.	git quiltimport	15
64.	git rerere.....	15

1. git init

Descripción: Inicializa un nuevo repositorio Git en el directorio actual.

```
Uso: git init
```

Explicación: Este comando crea una nueva carpeta oculta llamada “.git” en el directorio actual, que contiene toda la información necesaria para el control de versiones.

2. git clone

Descripción: Clona un repositorio existente desde una URL.

```
Uso: git clone <URL>
```

Explicación: Este comando descarga una copia completa de un repositorio remoto en tu máquina local. Por ejemplo, `git clone https://github.com/usuario/repositorio.git`.

3. git add

Descripción: Añade cambios al área de preparación (staging area).

```
Uso: git add <archivo> o git add .
```

Explicación: Este comando agrega cambios específicos (o todos los cambios) al área de preparación, listos para ser confirmados en el próximo commit. Por ejemplo, `git add archivo.txt` o `git add .` para agregar todos los cambios.

4. git commit

Descripción: Confirma los cambios en el repositorio.

```
Uso: git commit -m "Mensaje del commit"
```

Explicación: Este comando toma los cambios que están en el área de preparación y los guarda en el historial del repositorio con un mensaje descriptivo. Por ejemplo, `git commit -m "Añadida funcionalidad de login"`.

5. git status

Descripción: Muestra el estado actual del repositorio.

```
Uso: git status
```

Explicación: Este comando muestra qué archivos han sido modificados, cuáles están en el área de preparación y cuáles no están siendo rastreados por Git.

6. git push

Descripción: Envía los cambios confirmados al repositorio remoto.

Uso: git push <remoto> <rama>

Explicación: Este comando sube los commits locales a un repositorio remoto. Por ejemplo, git push origin main sube los cambios a la rama main del repositorio remoto origin.

7. git pull

Descripción: Obtiene los cambios más recientes del repositorio remoto y los fusiona con la rama actual.

Uso: git pull <remoto> <rama>

Explicación: Este comando es una combinación de git fetch (que descarga los cambios) y git merge (que fusiona los cambios en la rama actual). Por ejemplo, git pull origin main.

8. git branch

Descripción: Lista, crea o borra ramas.

Uso:

git branch (lista las ramas locales)

git branch -a (lista las ramas locales y remotos)

git branch <nombre-rama> (crea una nueva rama)

git branch -d <nombre-rama> (borra una rama)

Explicación: Este comando permite gestionar las ramas del repositorio. Las ramas son útiles para trabajar en características o correcciones de manera aislada.

9. git checkout

Descripción: Cambia de rama o restaura archivos.

Uso:

git checkout <nombre-rama> (cambia a la rama especificada)

git checkout -b <nombre-rama> (crea una nueva rama y se cambia a ella)

Explicación: Este comando te permite moverte entre ramas o restaurar archivos a un estado anterior.

10. git merge

Descripción: Fusiona una rama con la rama actual.

```
Uso: git merge <nombre-rama>
```

Explicación: Este comando combina los cambios de la rama especificada con la rama actual. Por ejemplo, si estás en la rama main y ejecutas git merge feature, fusionarás los cambios de feature en main.

11. git log

Descripción: Muestra el historial de commits.

```
Uso: git log
```

Explicación: Este comando muestra una lista de commits realizados en la rama actual, con detalles como el autor, la fecha y el mensaje del commit.

12. git fetch

Descripción: Descarga cambios del repositorio remoto, pero no los fusiona.

```
Uso: git fetch <remoto>
```

Explicación: Este comando obtiene los últimos cambios del repositorio remoto, pero no los aplica a tu rama actual. Es útil para ver qué cambios hay en el remoto antes de fusionarlos.

13. git reset

Descripción: Deshace cambios en el área de preparación o en el repositorio.

Uso:

```
git reset <archivo> (saca un archivo del área de preparación)
```

```
git reset --hard <commit> (restaura el repositorio a un commit específico,  
descartando todos los cambios posteriores)
```

Explicación: Este comando es poderoso y puede deshacer cambios en diferentes niveles, dependiendo de las opciones que uses.

14. git stash

Descripción: Guarda temporalmente cambios no confirmados.

Uso:

`git stash` (guarda los cambios)

`git stash apply` (aplica los cambios guardados)

Explicación: Este comando es útil cuando necesitas cambiar de rama pero no quieres confirmar los cambios actuales. Los cambios se guardan temporalmente y pueden ser recuperados más tarde.

15. git remote

Descripción: Gestiona conexiones a repositorios remotos.

Uso:

`git remote -v` (lista los repositorios remotos)

`git remote add <nombre> <URL>` (añade un nuevo repositorio remoto)

Explicación: Este comando te permite gestionar las conexiones a repositorios remotos, como GitHub o GitLab.

16. git diff

Descripción: Muestra diferencias entre commits, ramas, o el área de trabajo.

Uso:

`git diff` (muestra cambios no preparados)

`git diff --cached` (muestra cambios preparados)

Explicación: Este comando es útil para ver qué cambios se han realizado antes de confirmarlos.

17. git tag

Descripción: Crea, lista o borra etiquetas (tags) en el repositorio.

Uso:

`git tag <nombre-tag>` (crea un tag en el commit actual)

`git tag -a <nombre-tag> -m "Mensaje"` (crea un tag anotado)

Explicación: Los tags son útiles para marcar puntos específicos en el historial, como versiones de lanzamiento.

18. git rebase

Descripción: Reaplica commits sobre otra rama.

```
Uso: git rebase <rama>
```

Explicación: Este comando es similar a git merge, pero en lugar de fusionar, reaplica los commits de la rama actual sobre la rama especificada. Es útil para mantener un historial lineal.

19. git cherry-pick

Descripción: Aplica un commit específico de otra rama a la rama actual.

```
Uso: git cherry-pick <commit-hash>
```

Explicación: Este comando es útil cuando quieres aplicar un commit específico de otra rama sin fusionar toda la rama.

20. git rm

Descripción: Elimina archivos del repositorio y del área de trabajo.

```
Uso: git rm <archivo>
```

Explicación: Este comando elimina archivos y los marca como eliminados en el próximo commit.

21. git config

Descripción: Configura opciones de Git.

Uso:

```
git config --global user.name "Tu Nombre" (configura el nombre del usuario)
```

```
git config --global user.email "tu@email.com" (configura el email del usuario)
```

Explicación: Este comando te permite configurar opciones globales o locales de Git, como el nombre y el email del usuario.

22. git show

Descripción: Muestra información sobre un objeto Git (commit, tag, etc.).

```
Uso: git show <commit-hash>
```

Explicación: Este comando muestra detalles sobre un commit específico, como los cambios realizados.

23. git revert

Descripción: Deshace un commit creando un nuevo commit que revierte los cambios.

Uso: git revert <commit-hash>

Explicación: Este comando es útil para deshacer cambios de manera segura, ya que crea un nuevo commit que revierte los cambios del commit especificado.

24. git clean

Descripción: Elimina archivos no rastreados del directorio de trabajo.

Uso: git clean -f

Explicación: Este comando es útil para limpiar archivos que no están siendo rastreados por Git.

25. git blame

Descripción: Muestra quién modificó cada línea de un archivo.

Uso: git blame <archivo>

Explicación: Este comando es útil para rastrear quién hizo cambios específicos en un archivo.

26. git bisect

Descripción: Ayuda a encontrar el commit que introdujo un error.

Uso: git bisect start y luego git bisect good <commit> / git bisect bad <commit>

Explicación: Este comando es útil para realizar una búsqueda binaria en el historial de commits para encontrar el commit que introdujo un error.

27. git reflog

Descripción: Muestra un registro de todos los cambios en las referencias (ramas, HEAD, etc.).

Uso: git reflog

Explicación: Este comando es útil para recuperar commits perdidos o deshechos.

28. git submodule

Descripción: Gestiona submodulos dentro de un repositorio.

Uso:

```
git submodule add <URL> (añade un submodulo)
```

```
git submodule update (actualiza submodulos)
```

Explicación: Los submodulos son repositorios dentro de otros repositorios, útiles para incluir dependencias externas.

29. git worktree

Descripción: Gestiona múltiples árboles de trabajo en un solo repositorio.

Uso: `git worktree add <ruta> <rama>`

Explicación: Este comando permite trabajar en múltiples ramas simultáneamente en diferentes directorios.

30. git archive

Descripción: Crea un archivo comprimido del repositorio.

Uso: `git archive --format=zip --output=archivo.zip <rama>`

Explicación: Este comando es útil para crear un archivo comprimido de un repositorio o una rama específica.

31. git grep

Descripción: Busca texto en el repositorio.

Uso: `git grep "texto"`

Explicación: Este comando es útil para buscar texto en los archivos del repositorio.

32. git fsck

Descripción: Verifica la integridad del repositorio.

Uso: `git fsck`

Explicación: Este comando es útil para detectar problemas de integridad en el repositorio.

33. git gc

Descripción: Realiza una limpieza y optimización del repositorio.

Uso: git gc

Explicación: Este comando es útil para optimizar el repositorio y liberar espacio.

34. git shortlog

Descripción: Muestra un resumen de los commits por autor.

Uso: git shortlog

Explicación: Este comando es útil para obtener un resumen de los commits agrupados por autor.

35. git describe

Descripción: Describe el commit actual basado en tags.

Uso: git describe

Explicación: Este comando es útil para obtener una descripción del commit actual basada en los tags más cercanos.

36. git apply

Descripción: Aplica un parche al repositorio.

Uso: git apply <parche>

Explicación: Este comando es útil para aplicar parches generados con git diff.

37. git patch-id

Descripción: Calcula el ID de un parche.

Uso: git patch-id <parche>

Explicación: Este comando es útil para identificar parches de manera única.

38. git request-pull

Descripción: Genera un mensaje de solicitud de pull.

Uso: git request-pull <commit> <URL>

Explicación: Este comando es útil para generar un mensaje de solicitud de pull para enviar a un mantenedor de un repositorio.

39. git instaweb

Descripción: Inicia un servidor web para navegar por el repositorio.

Uso: git instaweb

Explicación: Este comando es útil para navegar por el repositorio en un navegador web.

40. git help

Descripción: Muestra ayuda sobre comandos de Git.

Uso: git help <comando>

Explicación: Este comando es útil para obtener información detallada sobre cualquier comando de Git.

41. git whatchanged

Descripción: Muestra los cambios recientes en el repositorio.

Uso: git whatchanged

Explicación: Este comando es similar a git log, pero muestra los cambios en los archivos.

42. git count-objects

Descripción: Cuenta objetos en el repositorio.

Uso: git count-objects

Explicación: Este comando es útil para obtener estadísticas sobre los objetos en el repositorio.

43. git verify-commit

Descripción: Verifica la firma GPG de un commit.

Uso: git verify-commit <commit>

Explicación: Este comando es útil para verificar la autenticidad de un commit firmado.

44. git verify-tag

Descripción: Verifica la firma GPG de un tag.

Uso: git verify-tag <tag>

Explicación: Este comando es útil para verificar la autenticidad de un tag firmado.

45. git merge-base

Descripción: Encuentra la base común de dos ramas.

Uso: git merge-base <rama1> <rama2>

Explicación: Este comando es útil para encontrar el commit común más reciente entre dos ramas.

46. git notes

Descripción: Añade o muestra notas en los commits.

Uso:

git notes add <commit> (añade una nota)

git notes show <commit> (muestra una nota)

Explicación: Este comando es útil para añadir información adicional a los commits.

47. git replace

Descripción: Reemplaza objetos en el repositorio.

Uso: git replace <objeto> <nuevo-objeto>

Explicación: Este comando es útil para reemplazar objetos en el repositorio sin modificar el historial.

48. git rev-parse

Descripción: Parsea y muestra información sobre objetos Git.

Uso: git rev-parse <objeto>

Explicación: Este comando es útil para obtener información detallada sobre objetos Git.

49. git symbolic-ref

Descripción: Lee o modifica referencias simbólicas.

Uso:

git symbolic-ref HEAD (lee la referencia simbólica)

git symbolic-ref HEAD refs/heads/nueva-rama (modifica la referencia simbólica)

Explicación: Este comando es útil para gestionar referencias simbólicas como HEAD.

50. git update-ref

Descripción: Actualiza referencias en el repositorio.

```
Uso: git update-ref <ref> <nuevo-valor>
```

Explicación: Este comando es útil para actualizar referencias de manera manual.

51. git filter-branch

Descripción: Reescribe el historial del repositorio.

```
Uso: git filter-branch --tree-filter 'comando'
```

Explicación: Este comando es útil para reescribir el historial del repositorio, como cambiar correos electrónicos o eliminar archivos.

52. git fast-export

Descripción: Exporta el repositorio a un formato legible.

```
Uso: git fast-export
```

Explicación: Este comando es útil para exportar el repositorio a un formato que puede ser importado por otros sistemas.

53. git fast-import

Descripción: Importa un repositorio desde un formato legible.

```
Uso: git fast-import
```

Explicación: Este comando es útil para importar un repositorio desde un formato generado por git fast-export.

54. git bundle

Descripción: Empaqueta el repositorio en un archivo.

```
Uso: git bundle create <archivo> <rama>
```

Explicación: Este comando es útil para empaquetar un repositorio en un archivo que puede ser compartido.

55. git daemon

Descripción: Inicia un servidor Git.

```
Uso: git daemon
```

Explicación: Este comando es útil para iniciar un servidor Git que puede ser accedido por otros.

56. git shell

Descripción: Inicia un shell restringido para Git.

Uso: git shell

Explicación: Este comando es útil para proporcionar acceso restringido a un repositorio Git.

57. git svn

Descripción: Interactúa con un repositorio Subversion (SVN).

Uso:

git svn clone <URL> (clona un repositorio SVN)

git svn rebase (actualiza el repositorio Git con cambios de SVN)

Explicación: Este comando es útil para trabajar con repositorios SVN desde Git.

58. git annex

Descripción: Gestiona archivos grandes en Git.

Uso: git annex

Explicación: Este comando es útil para gestionar archivos grandes que no deben ser almacenados directamente en el repositorio Git.

59. git lfs

Descripción: Gestiona archivos grandes con Git LFS (Large File Storage).

Uso:

git lfs install (instala Git LFS)

git lfs track <patrón> (rastrea archivos grandes)

Explicación: Este comando es útil para gestionar archivos grandes en repositorios Git.

60. git flow

Descripción: Implementa un flujo de trabajo Git.

Uso:

git flow init (inicializa Git Flow)

git flow feature start <nombre> (inicia una nueva característica)

Explicación: Este comando es útil para implementar un flujo de trabajo Git basado en ramas.

61. git subrepo

Descripción: Gestiona subrepositorios en Git.

Uso:

git subrepo clone <URL> (clona un subrepositorio)

git subrepo pull (actualiza un subrepositorio)

Explicación: Este comando es útil para gestionar subrepositorios dentro de un repositorio Git.

62. git absorb

Descripción: Absorbe cambios en commits existentes.

Uso: git absorb

Explicación: Este comando es útil para absorber cambios en commits existentes en lugar de crear nuevos commits.

63. git quiltimport

Descripción: Importa parches de Quilt a Git.

Uso: git quiltimport

Explicación: Este comando es útil para importar parches de Quilt a un repositorio Git.

64. git rerere

Descripción: Reutiliza resoluciones de conflictos.

Uso: git rerere

Explicación: Este comando es útil para reutilizar resoluciones de conflictos en fusiones futuras.

Este resumen cubre los comandos más comunes y útiles de Git. Con estos comandos, podrás gestionar eficientemente tus proyectos y colaborar con otros desarrolladores. ¡Espero que te sea de ayuda!